# Code Verification in Rust with Verus

## From intended behavior to machine-checked contracts

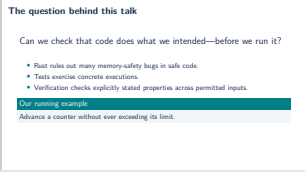
# The question behind this talk

Can we check that code does what we intended—before we run it?

- Rust rules out many memory-safety bugs in safe code.
- Tests exercise concrete executions.
- Verification checks explicitly stated properties across permitted inputs.

**Our running example**

Advance a counter without ever exceeding its limit.




Audience: developers comfortable reading Rust; no formal-methods background assumed. Aim for about 30 minutes plus questions. Ask the audience what evidence they usually require before trusting a small generated function.

# The route

Introduction

What is code verification

Z3

Verus

Verification as LLM Guardrails

**The route**

Introduction

What is code verification

Z3

Verus

Verification as LLM Guardrails

Timing: introduction 3 minutes; verification 5; Z3 5; Verus 10; LLM guardrails 7. The same counter example connects the sections.

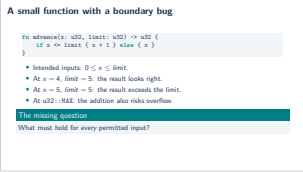
# A small function with a boundary bug

```
fn advance(x: u32, limit: u32) -> u32 {
    if x <= limit { x + 1 } else { x }
}
```

- Intended inputs: $0 \leq x \leq \mathit{limit}$.
- At $x = 4$, $\mathit{limit} = 5$: the result looks right.
- At $x = 5$, $\mathit{limit} = 5$: the result exceeds the limit.
- At `u32::MAX`: the addition also risks overflow.

**The missing question**

What must hold for *every* permitted input?




Do not frame this as a failure of testing: a boundary test catches it immediately. The point is to make the intended behavior explicit and check the whole allowed input domain. Rust's type checker accepts this code. Overflow behavior depends on build settings.

# Turn intent into a contract

**Precondition: caller's obligation**

$x \leq limit$

**Postcondition: callee's promise**

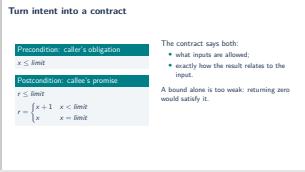
$r \leq limit$

$$r = \begin{cases} x + 1 & x < limit \\ x & x = limit \end{cases}$$

The contract says both:

- what inputs are allowed;
- exactly how the result relates to the input.

A bound alone is too weak: returning zero would satisfy it.




These equations use mathematical integers. Later, Verus must additionally establish that executable u32 arithmetic is safe. Preconditions restrict the promise; they do not validate external input at runtime.

# What does a proof establish?

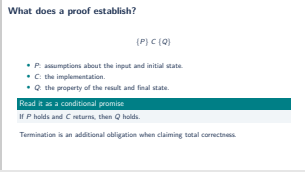
$$\{P\}\ C\ \{Q\}$$

- $P$: assumptions about the input and initial state.
- $C$: the implementation.
- $Q$: the property of the result and final state.

**Read it as a conditional promise**

If $P$ holds and $C$ returns, then $Q$ holds.

Termination is an additional obligation when claiming total correctness.




This is partial correctness. Avoid saying that all verified programs terminate automatically. Safety obligations, such as preventing overflow, are additional checks generated by a verifier.

# Different tools provide different evidence

| Tool | Evidence | Boundary |
|---|---|---|
| Rust type checker | Types, ownership, borrowing | Does not establish arbitrary business rules |
| Tests / fuzzing | Behavior on explored executions | Unexplored inputs remain |
| Deductive verification | Contract holds over modeled inputs | Depends on specification and trusted assumptions |

Keep all three: each answers a useful question.

**Different tools provide different evidence**

| Tool | Evidence | Boundary |
|---|---|---|
| Rust type checker | Types, ownership, borrowing | Does not establish arbitrary business rules |
| Tests / fuzzing | Behavior on explored executions | Unexplored inputs remain |
| Deductive verification | Contract holds over modeled inputs | Depends on specification and trusted assumptions |

Keep all three: each answers a useful question.

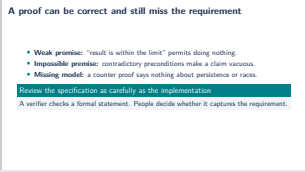
Verification does not replace integration tests, performance measurements, or checking the real deployment environment. Safe Rust's guarantees also depend on the soundness of unsafe implementations underneath it.

# A proof can be correct and still miss the requirement

- **Weak promise:** "result is within the limit" permits doing nothing.
- **Impossible premise:** contradictory preconditions make a claim vacuous.
- **Missing model:** a counter proof says nothing about persistence or races.

**Review the specification as carefully as the implementation**

A verifier checks a formal statement. People decide whether it captures the requirement.




Use the counter to make this concrete: a product requirement may include atomic updates across threads. Our pure function models no shared state, so its contract cannot establish that property.

# Z3 checks logical constraints

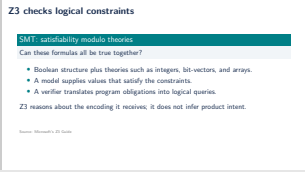
**SMT: satisfiability modulo theories**

Can these formulas all be true together?

- Boolean structure plus theories such as integers, bit-vectors, and arrays.
- A model supplies values that satisfy the constraints.
- A verifier translates program obligations into logical queries.

Z3 reasons about the encoding it receives; it does not infer product intent.

Source: Microsoft's Z3 Guide




Use mathematical integers for the teaching query. Fixed-width arithmetic requires explicit range constraints or a bit-vector model; integers alone do not model Rust overflow.

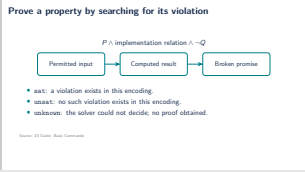
# Prove a property by searching for its violation

$$P \wedge \text{implementation relation} \wedge \neg Q$$




- `sat`: a violation exists in this encoding.
- `unsat`: no such violation exists in this encoding.
- `unknown`: the solver could not decide; no proof obtained.

Source: Z3 Guide: Basic Commands




A timeout also leaves the obligation unproved. With abstractions, a satisfying model may require interpretation before calling it a real program counterexample. Our small exact arithmetic example has a direct interpretation.

# Ask Z3 for the boundary bug

```
(set-logic QF_LIA)
(declare-const x Int)
(declare-const limit Int)
(assert (and (<= 0 x) (<= x limit)
             (<= limit 4294967295)))
(define-fun r () Int (ite (<= x limit) (+ x 1) x))
(assert (> r limit)) ; violate the bound
(check-sat)
(get-model)
```

`sat`: for example, $x = 0$, $\mathit{limit} = 0$ gives $r = 1$.

**Ask Z3 for the boundary bug**

```
(set-logic QF_LIA)
(declare-const x Int)
(declare-const limit Int)
(assert (and (<= 0 x) (<= x limit)
             (<= limit 4294967296)))
(define-fun r () Int (ite (<= x limit) (+ x 1) x))
(assert (> r limit)) ; violate the bound
(check-sat)
(get-model)
```

sat: for example, $x = 0$, $limit = 0$ gives $r = 1$.

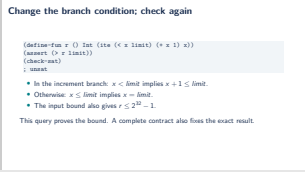
This is examples/counter-bug.smt2. The exact model printed by Z3 may differ. The chosen witness is valid regardless of which model the solver chooses.

# Change the branch condition; check again

```
(define-fun r () Int (ite (< x limit) (+ x 1) x))
(assert (> r limit))
(check-sat)
; unsat
```

- In the increment branch: $x < \mathit{limit}$ implies $x + 1 \leq \mathit{limit}$.
- Otherwise: $x \leq \mathit{limit}$ implies $x = \mathit{limit}$.
- The input bound also gives $r \leq 2^{32} - 1$.

This query proves the bound. A complete contract also fixes the exact result.




The full standalone query is in examples/counter-fixed.smt2. It uses the same input assertions as the previous slide. Verus automates generation of obligations from the source instead of requiring us to manually maintain this encoding.

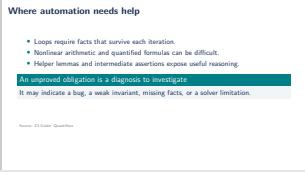
# Where automation needs help

- Loops require facts that survive each iteration.
- Nonlinear arithmetic and quantified formulas can be difficult.
- Helper lemmas and intermediate assertions expose useful reasoning.

**An unproved obligation is a diagnosis to investigate**

It may indicate a bug, a weak invariant, missing facts, or a solver limitation.

Source: Z3 Guide: Quantifiers




Avoid equating proof failure with a concrete bug. Break a difficult claim into smaller claims before increasing solver limits. Quantifier instantiation is heuristic in general.

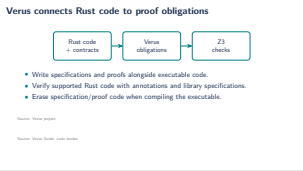
# Verus connects Rust code to proof obligations




- Write specifications and proofs alongside executable code.
- Verify supported Rust code with annotations and library specifications.
- Erase specification/proof code when compiling the executable.

Source: Verus project

Source: Verus Guide: code modes



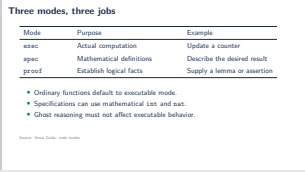

Verus supports a subset of Rust; existing crates are not automatically fully verified. The diagram is conceptual, not a complete description of the internal compiler pipeline.

# Three modes, three jobs

| Mode | Purpose | Example |
|---|---|---|
| `exec` | Actual computation | Update a counter |
| `spec` | Mathematical definitions | Describe the desired result |
| `proof` | Establish logical facts | Supply a lemma or assertion |

- Ordinary functions default to executable mode.
- Specifications can use mathematical `int` and `nat`.
- Ghost reasoning must not affect executable behavior.

Source: Verus Guide: code modes



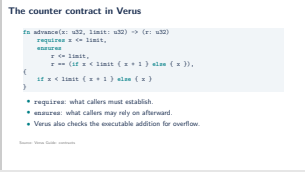

Machine integers still matter in exec code. Mathematical types let the specification describe arithmetic without silently wrapping. Proof functions can package reusable arguments.

# The counter contract in Verus

```
fn advance(x: u32, limit: u32) -> (r: u32)
    requires x <= limit,
    ensures
        r <= limit,
        r == (if x < limit { x + 1 } else { x }),
{
    if x < limit { x + 1 } else { x }
}
```

- `requires`: what callers must establish.
- `ensures`: what callers may rely on afterward.
- Verus also checks the executable addition for overflow.

Source: Verus Guide: contracts



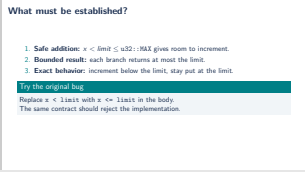

The complete file adds use vstd::prelude::* and a verus! block. In specification expressions, the addition is mathematical. In the executable branch, $x < \text{limit}$ and the u32 bound on limit justify $x + 1$. This is intended to be an automatically discharged example.

# What must be established?

1. **Safe addition:** $x < \mathit{limit} \leq$ `u32::MAX` gives room to increment.
2. **Bounded result:** each branch returns at most the limit.
3. **Exact behavior:** increment below the limit, stay put at the limit.

## Try the original bug

Replace `x < limit` with `x <= limit` in the body.
The same contract should reject the implementation.




Keep the specification unchanged during this mutation. At equality the body disagrees with the exact postcondition and breaks the bound; at MAX the body can overflow as well. This is a useful demonstration even when not running live.

# Callers use the contract

```
fn client() {
    let a = advance(4, 5);
    assert(a == 5);
    let b = advance(5, 5);
    assert(b == 5);
}
```

A call to `advance(6, 5)` fails its precondition obligation.

**Compositional reasoning**

Verify a function once; reason about its callers through its contract.



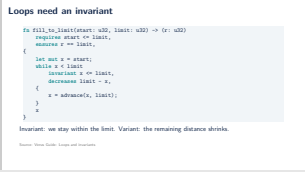

Assertions here are verification assertions, not runtime input validation. An application accepting untrusted inputs still needs executable checks before calling a function whose precondition might not hold.

# Loops need an invariant

```
fn fill_to_limit(start: u32, limit: u32) -> (r: u32)
    requires start <= limit,
    ensures r == limit,
{
    let mut x = start;
    while x < limit
        invariant x <= limit,
        decreases limit - x,
    {
        x = advance(x, limit);
    }
    x
}
```

Invariant: we stay within the limit. Variant: the remaining distance shrinks.

Source: Verus Guide: Loops and invariants



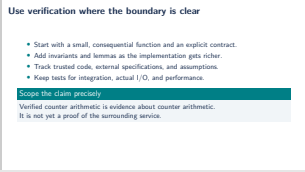

Deliberately simple, not a recommended implementation of assignment. Establish the invariant initially, preserve it after each step, then combine it with the negated guard to get equality. The variant is nonnegative and strictly decreases while the guard holds.

# Use verification where the boundary is clear

- Start with a small, consequential function and an explicit contract.
- Add invariants and lemmas as the implementation gets richer.
- Track trusted code, external specifications, and assumptions.
- Keep tests for integration, actual I/O, and performance.

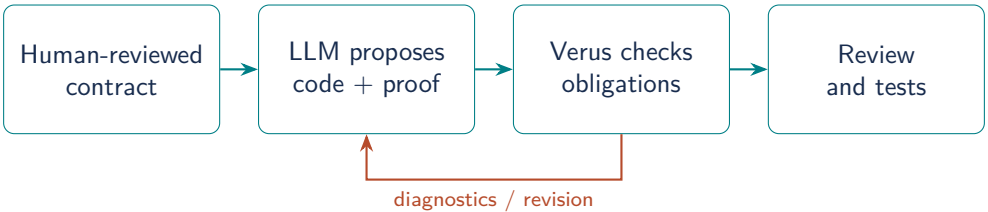
**Scope the claim precisely**

Verified counter arithmetic is evidence about counter arithmetic.
It is not yet a proof of the surrounding service.




The trust boundary includes the verifier, solver, compilation chain, and any assumed or externally specified behavior. Proof engineering is real work; select components where the property and payoff are clear.

# Let the model propose; require checked evidence




The acceptance signal is a verifier result on the submitted artifact.



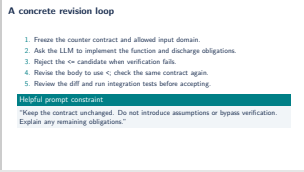

This is a proposed engineering workflow, not a claim that Verus guarantees all LLM output is safe. LLMs can help draft implementations, invariants, and lemmas; the generated proof text itself is not evidence until checked.

# A concrete revision loop

1. Freeze the counter contract and allowed input domain.
2. Ask the LLM to implement the function and discharge obligations.
3. Reject the <= candidate when verification fails.
4. Revise the body to use <; check the same contract again.
5. Review the diff and run integration tests before accepting.

**Helpful prompt constraint**

"Keep the contract unchanged. Do not introduce assumptions or bypass verification. Explain any remaining obligations."




A prompt constraint helps communicate intent but is not enforcement. Repository permissions, review, and CI must enforce the acceptance policy independently. Do not let an agent edit the verifier command to make a failed run appear successful.

# Protect the meaning of a green check

- **Lock the target:** review contract changes separately from code fixes.
- **Audit escape hatches:** new assumptions or trusted bodies require review.
- **Check coverage:** verify the intended functions and exact revision.
- **Fail closed:** errors, skipped checks, and timeouts are not passes.
- **Preserve evidence:** record tool versions, commands, and results.

**Protect the meaning of a green check**

- **Lock the target:** review contract changes separately from code fixes.
- **Audit escape hatches:** new assumptions or trusted bodies require review.
- **Check coverage:** verify the intended functions and exact revision.
- **Fail closed:** errors, skipped checks, and timeouts are not passes.
- **Preserve evidence:** record tool versions, commands, and results.

For example, adding assume(false) can make subsequent claims vacuous. A verifier accepting code is useful only if the acceptance path checks the right claim without expanding the trusted surface unnoticed. This slide describes a recommended policy, not a built-in Verus CI feature.

# The guardrail has a defined reach

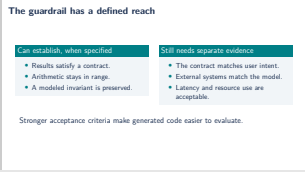
## Can establish, when specified

- Results satisfy a contract.
- Arithmetic stays in range.
- A modeled invariant is preserved.

## Still needs separate evidence

- The contract matches user intent.
- External systems match the model.
- Latency and resource use are acceptable.

Stronger acceptance criteria make generated code easier to evaluate.

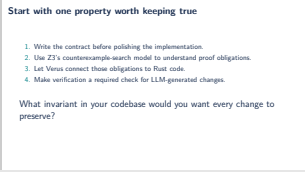



Verification is especially valuable when humans can describe the invariant more reliably than they can inspect every generated implementation. Properties omitted from the contract remain outside the claim.

# Start with one property worth keeping true

1. Write the contract before polishing the implementation.
2. Use Z3's counterexample-search model to understand proof obligations.
3. Let Verus connect those obligations to Rust code.
4. Make verification a required check for LLM-generated changes.

What invariant in your codebase would you want every change to preserve?




Invite questions. Suggested discussion: bounds in a parser, monotonic sequence numbers, or a state transition that must preserve an authorization invariant. Each requires its own explicit model.

# References and demo files

- Verus Tutorial and Reference

  `https://verus-lang.github.io/verus/guide/`

- Verus source and installation instructions

  `https://github.com/verus-lang/verus`

- Microsoft's Z3 Guide

  `https://microsoft.github.io/z3guide/`

Local examples: `examples/counter.rs`, `examples/counter-bug.smt2`, `examples/counter-fixed.smt2`.




The README documents build commands and validation status. Source links throughout the deck are clickable. Speaker notes live in the TeX source and are hidden in the audience PDF.